

# 05 Agent 开卷速查

## 05 Agent 开卷速查

### 考场定位

Agent 的考试重点来自三份 tutorial PPT：

- 1. tutorial-code-agent.pptx：Code Agent 怎么用、怎么抽象。
- 2. tutorial-long-run-agent.pptx：Long-run Agent，reactive -> proactive。
- 3. tutorial-agent-memory.pptx：Agent Memory，持久化记忆和 memory system。

复习目标不是背项目命令，而是会解释：

Agent = LLM + Harness  
Loop / Tool / Skill / Memory 为什么构成 Agent  
Code Agent 和 Long-run Agent 有什么区别  
Agent Memory 怎么存、写、取、更新、淘汰  
AgentLab / mini-agent-demo 如何对应这些概念

### 目录

没有页码时，直接用“知识点编号”定位下面对应小节；考场遇到题目关键词，先按本表跳转，再套用第 18 节简答模板。

| 知识点编号    | 标题                  | 常见题目关键词              | 必写词                          |
|----------|---------------------|----------------------|------------------------------|
| <u>1</u> | 核心模型                | Agent 是什么；为什么不只是 LLM | LLM + Harness，goal，loop      |
| <u>2</u> | Loop / Tool / Skill | Agent 如何行动；工具和       | main loop，tool list，skill.md |

| 知识点编号     | 标题                           | 常见题目关键词                             | 必写词                                                           |
|-----------|------------------------------|-------------------------------------|---------------------------------------------------------------|
|           |                              | 技能                                  |                                                               |
| <u>3</u>  | Code Agent 演进                | ChatBot 到 Agent; AI 编程形态变化          | ChatBot, Copilot, Agent Act                                   |
| <u>4</u>  | Code Agent 三种形态              | Tab / IDE / CLI 区别                  | Tab Complete, IDE Agent, CLI Agent                            |
| <u>5</u>  | Tool / Provider / Model 三层抽象 | 工具、平台、模型怎么区分                        | Tool, Provider/API/Billing, Model                             |
| <u>6</u>  | Prompt 模板                    | Prompt 怎么写; 如何让 Agent 闭环            | Goal, Context, Constraints, Output, Verification              |
| <u>7</u>  | Mini-Agent / AgentLab        | mini-agent-demo; JSON tool protocol | loop, tools, skill, memory, trace                             |
| <u>8</u>  | Long-run Agent               | Code Agent vs Long-run Agent        | reactive -> proactive                                         |
| <u>9</u>  | 时序能力: Cron / Heartbeat       | 定时任务; 主动唤醒; 外部消息                    | Cron, Heartbeat, Gateway                                      |
| <u>10</u> | Memory vs Context            | 记忆和上下文区别; 为什么需要持久记忆                 | context 有限, memory 持久                                         |
| <u>11</u> | Memory 管理五问                  | memory 怎么存、写、读、改、淘汰                 | Storage, Write, Retrieval, Update, Evict                      |
| <u>12</u> | 检索和淘汰策略                      | Retrieval / Eviction; 如何召回记忆        | keyword/vector/graph/hybrid, recency/frequency/relevance/heat |

| 知识点<br>编号 | 标题                    | 常见题目关键词                           | 必写词                                           |
|-----------|-----------------------|-----------------------------------|-----------------------------------------------|
| <u>13</u> | MemGPT                | virtual context; OS 类比            | main context, external context, function call |
| <u>14</u> | MemoryOS              | 分层 memory system                  | STM, MTM, LPM                                 |
| <u>15</u> | Mem0                  | 生产级 memory service; memory update | ADD/UPDATE/DELETE/NOOP                        |
| <u>16</u> | 自我进化: Learning Loop   | Agent 如何越用越聪明                     | memory -> skill -> optimize -> retrieve       |
| <u>17</u> | SubAgent / MultiAgent | 子任务分发; 多 Agent 协作                 | 并行, 工作看板, 输入输出, 校验                            |
| <u>18</u> | 简答模板                  | 考场直接套句                            | 按题目选 1-2 段改写                                  |
| <u>19</u> | 高频词                   | 最后快速扫关键词                          | 背关键词, 不背长段落                                   |
| <u>20</u> | 考场最后 20 秒             | 临交卷前补核心词                          | 总纲、形态、memory、三系统                              |

来源优先级: tutorial-code-agent.pptx / tutorial-long-run-agent.pptx / tutorial-agent-memory.pptx > ICS-AgentLab.pptx / AgentLab README > review.pdf

## 1. 核心模型

Agent = LLM + Harness

| 组成  | 作用                |
|-----|-------------------|
| LLM | 理解任务、推理、规划、生成行动意图 |

| 组成      | 作用                             |
|---------|--------------------------------|
| Harness | 维护上下文、暴露工具、执行动作、检查权限、记录过程、处理错误 |
| Goal    | Agent 围绕目标持续推进，不是一次性聊天         |
| Loop    | 观察 -> 思考 -> 行动 -> 再观察          |

一句话：

Agent 不是“会聊天的模型”，而是 LLM 在 harness 支撑下，围绕 goal 不断观察、决策、行动、反馈的系统。

为什么不能只有 LLM：

LLM 只能产生文本或行动意图；真正完成任务需要工具、状态、权限、执行环境、反馈闭环和持久记忆，这些由 harness 提供。

## 2. Loop / Tool / Skill

三件事是 Code Agent / Mini-Agent 的基础：

| 概念    | 课件说法                   | 考场解释                        |
|-------|------------------------|-----------------------------|
| Loop  | Agent 核心是交互式 main loop | 不断调用 LLM，解析行动，执行工具，回填结果     |
| Tool  | 调用外部工具，与环境交互           | bash、本地指令、MCP 外部工具、文件系统、API |
| Skill | 把特定领域能力/知识包装成 skill.md | 流程说明按需加载，不全部塞进 prompt       |

Loop 最小形态：

User Task -> LLM -> Action/Tool Call -> Tool Result -> LLM -> ... -> Final

AgentLab / mini-agent-demo 对应：

| 练习/实验          | 对应概念               |
|----------------|--------------------|
| s01-agent-loop | 实现 Agent main loop |
| s02-tool-use   | 实现文件操作 tools       |

| 练习/实验          | 对应概念                                       |
|----------------|--------------------------------------------|
| s03-load-skill | 编写 skill / 测试 / 修 bug                      |
| AgentLab       | loop + tool + skill + memory + trace 的综合实验 |

### 3. Code Agent 演进

课件主线：

ChatBot Answer -> Copilot Suggest -> Agent Act

| 阶段              | 输入    | 输出        | 关键词                              |
|-----------------|-------|-----------|----------------------------------|
| ChatBot Answer  | 一句问题  | 一段回答      | 搜索、问答、头脑风暴、Ctrl C/V 工程           |
| Copilot Suggest | 局部上下文 | 一小段代码建议   | 补全、改写、Tab 工程                     |
| Agent Act       | 一个任务  | 执行过程 + 结果 | 完整项目上下文、TODO、LOOP、TOOL、Prompt 工程 |

考场句：

Code Agent 的变化不是“回答更长”，而是从回答问题变成围绕任务执行动作：能看项目上下文、调用工具、多轮迭代并交付结果。

### 4. Code Agent 三种形态

Tab Complete >> IDE Agent >> CLI Agent

| 形态           | 输入           | 输出     | 优点       | 适合                |
|--------------|--------------|--------|----------|-------------------|
| Tab Complete | 局部上下文        | 短代码片段  | 快、自然、低风险 | 函数内、样板代码、微改动      |
| IDE Agent    | 自然语言 feature | 跨文件修改  | 开发小功能很快  | 小项目扩展、组件重构、bug 修复 |
| CLI Agent    | 仓库级任务        | 最终任务结果 | 执行与验证闭环  | 测试、搭建项目、仓库级任      |

| 形态 | 输入 | 输出 | 优点 | 适合 |
|----|----|----|----|----|
|    |    |    |    | 务  |

Tab Complete

要点：

- 根据当前文件和打开标签页给建议。
- 需要极高响应速度和快速上下文组装。
- Copilot 依赖当前/活动文件；定义文件也打开时更准。
- Trae CUE 会对 workspace 建本地索引，结合语言服务器和向量索引。

IDE Agent

要点：

- 基于当前代码仓库辅助开发。
- 可以快速指定上下文。
- 具备 Agent 的完整能力，但仍主要在 IDE 场景里工作。

CLI Agent

要点：

- 真正的 Vibe Coding：输入意图，让 Agent 操作项目。
- 典型工具：Codex CLI、Claude Code CLI、Hermes CLI。
- 课件句：Bash is all agents need。

IDE Agent vs CLI Agent：

| 维度   | IDE Agent        | CLI Agent           |
|------|------------------|---------------------|
| 权限边界 | 通常围绕 IDE / 打开的项目 | 能读写文件、跑命令、查网络       |
| 工具能力 | IDE 内补全、重构、对话    | shell、文件系统、终端、web 等 |
| 一句话  | AI 辅助你写代码        | AI 替你操作电脑           |

5. Tool / Provider / Model 三层抽象

课件强调不要把这些混在一起：

| 层        | 是什么                | 例子                                           |
|----------|--------------------|----------------------------------------------|
| Tool     | 具体 Agent 产品/运行环境   | Trae, Claude Code, OpenCode, Codex           |
| Provider | 模型提供商 / API / 计费平台 | OpenAI, Anthropic, DeepSeek, OpenRouter      |
| Model    | 实际被调用的模型           | DeepSeek V4, Claude Sonnet, GPT, hy3-preview |

常见区分：

- Tool：IDE Agent vs CLI Agent。
- Provider：模型提供商 vs 平台聚合商。
- 付费：API token vs Code Plan。
- Model：闭源顶级模型 vs 顶尖开源模型。

考场句：

Agent 工具、模型提供商和模型本身是三层抽象：工具负责交互和 harness，provider 负责 API/计费/路由，model 负责生成和推理。

## 6. Prompt 模板

课件 prompt template：

请基于当前项目完成 <Goal>。  
约束：<Constraints>。  
最终请 <Output>，  
并用 <Verification> 验证。

五要素：

| 要素           | 问题           | 写法              |
|--------------|--------------|-----------------|
| Goal         | 到底要完成什么      | 明确任务目标          |
| Context      | 正在读哪个项目/文件   | 指定仓库、模块、相关文件    |
| Constraints  | 不要改什么/不要引入什么 | 限制范围、依赖、风格      |
| Output       | 希望交付什么       | 代码、文档、测试、总结     |
| Verification | 怎么判断做对了      | 跑测试、编译、截图、人工检查点 |

考场句：

好 prompt 不是多写废话，而是把 goal、context、constraints、output 和 verification 交代清楚，让 Agent 能闭环执行。

## 7. Mini-Agent / AgentLab

AgentLab 和 mini-agent-demo 的意义：

让你看懂并复现一个真正的 Agent：它不是一次调用 LLM，而是 loop + tool + skill + memory + trace 的组合。

AgentLab 设计对应概念：

| 设计                      | Agent 概念          | 解释                             |
|-------------------------|-------------------|--------------------------------|
| 普通 Chat Completion      | LLM 推理            | 不依赖 SDK 自带 Agent 框架            |
| 禁用 native tool calling  | harness 控制协议      | 自己实现 tool protocol             |
| JSON: tool_call / final | structured action | 区分行动和最终回答                      |
| parse_error             | error recovery    | 格式错了要修复                        |
| tools                   | action space      | 文件、shell、skill、memory、subagent |
| load_skill              | on-demand skill   | 需要时读流程                         |
| save_memory/read_memory | persistent memory | 跨 session 记忆                   |
| trace                   | observability     | 记录执行证据链                        |

JSON 协议例子：

```
{"type": "tool_call", "name": "read_file", "arguments": {"path": "a.txt"}}
{"type": "final", "content": "answer for the user"}
```

为什么要手写 JSON 协议：

- 把自然语言意图变成结构化 action。
- harness 能解析工具名和参数。
- 能限制 Agent 的行动格式。
- 方便记录 trace 和处理错误。

## 8. Long-run Agent

Code Agent vs Long-run Agent:

| 维度   | Code Agent    | Long-run Agent  |
|------|---------------|-----------------|
| 交互方式 | 人发任务，Agent 响应 | Agent 可定时/被消息唤醒 |
| 时间尺度 | 单轮/多轮任务       | 长期运行            |
| 状态   | 会话级，退出易丢      | 持久状态            |
| 主动性  | reactive，被动响应 | proactive，主动执行  |
| 典型能力 | 写代码、跑测试、修 bug | 监控、提醒、长期任务、个人助理 |

课件核心转变：

reactive -> proactive

Long-run Agent 四个关键能力：

| 能力                | 含义                  |
|-------------------|---------------------|
| Cron / Heartbeat  | 定时醒来或定期检查自己该做什么     |
| Persistent Memory | 保存长期状态和用户偏好         |
| Self Improvement  | 沉淀重复流程，修正旧经验        |
| Message / Gateway | 接收外部请求，如 IM、网页、服务入口 |

## 9. 时序能力：Cron / Heartbeat

| 机制        | 含义             | 例子                |
|-----------|----------------|-------------------|
| Cron      | 按时间表自动醒来执行指定任务 | 每天生成技术晨报          |
| Heartbeat | 定期醒来做“清醒检查”    | 检查 tasks 列表，推自己一把 |
| Gateway   | 外部消息入口/路由      | 微信、QQ、飞书、Telegram |

Cron vs Heartbeat:

- Cron 更像定时任务：到点执行。
- Heartbeat 更像自检机制：醒来后检查有没有该继续推进的事。
- Gateway 让 Agent 不只在命令行里工作，而能接收外部请求。

考场句：

Long-run Agent 需要时序能力：Cron 让它按时执行，Heartbeat 让它周期性检查状态，Gateway 让它接收外部消息。

## 10. Memory vs Context

课件定义：

| 项       | 含义         | 特点                   |
|---------|------------|----------------------|
| Context | 本轮推理时的上下文  | 容量有限，直接影响当前回答        |
| Memory  | 跨会话保存的持久信息 | 容量更大，需要检索后放回 context |

如果 Long-run Agent 没有 memory：

- 不同 session 反复问同样问题。
- 过去犯的错下次还会犯。
- 无法完成长期任务。
- 无法保持个性化。

Memory 记什么：

| 类型 | 内容              |
|----|-----------------|
| 事实 | 用户偏好、项目背景、长期目标  |
| 状态 | 任务进度、环境配置、上下文摘要 |
| 经验 | 踩坑记录、固定约定、可复用判断 |

一句话：

Memory 是通过外部系统保存关键信息，并在未来需要时重新检索、放回 context。

## 11. Memory 管理五问

持久化管理需要解决 5 个问题：

| 问题        | 含义        | 考场写法                 |
|-----------|-----------|----------------------|
| Storage   | 写到哪里      | 文件、数据库、搜索索引、raw logs |
| Write     | 什么时候写、怎么写 | 事实/状态/经验变重要时写入       |
| Retrieval | 什么时候读     | 当前 context 缺少所需知识时检索 |

| 问题     | 含义               | 考场写法           |
|--------|------------------|----------------|
| Update | 如何合并和纠错          | 新事实覆盖旧事实，冲突要处理 |
| Evict  | 空间/context 满了怎么办 | 压缩、淘汰、晋升、迁移    |

Agent Memory 存储层级：

| 层级                      | 作用        |
|-------------------------|-----------|
| Prompt / Active Context | 当前直接可见的信息 |
| Session History         | 当前或近期会话   |
| User / Project Memory   | 用户偏好、项目事实 |
| Raw Logs                | 原始历史记录    |
| Database / Search Index | 检索和索引用    |

## 12. 检索和淘汰策略

### Retrieval

Agent Memory Retrieval 的几种方式：

| 策略              | 含义      | 优缺点            |
|-----------------|---------|----------------|
| Always inject   | 固定注入    | 简单，但浪费 context |
| Keyword search  | 关键词匹配   | 快，语义能力弱        |
| Vector search   | 语义相似度召回 | 能找近义内容，但可能误召回  |
| Graph traversal | 关系推理    | 适合实体/关系复杂场景    |
| Hybrid search   | 组合检索    | 更稳，但系统复杂       |

检索触发：

当前上下文没有需要的知识时，从 memory storage 查找相关 item，并注入 active context。

### Eviction

Agent Context Eviction：

| 策略             | 含义     |
|----------------|--------|
| Sliding Window | 保留最近窗口 |

| 策略        | 含义           |
|-----------|--------------|
| Recency   | 最近使用的优先保留    |
| Frequency | 经常被用的优先保留    |
| Relevance | 和当前任务相关的优先保留 |
| Heat      | 综合热度决定晋升或淘汰  |

一句话：

Retrieval 决定该把哪些长期信息调入 context；Eviction 决定 context 满时哪些内容保留、压缩、迁移或淘汰。

### 13. MemGPT

核心问题：

LLM 有固定长度的 context window；长期对话和大量外部知识会超出窗口。

MemGPT 核心思想：

virtual context management

类比：

| OS                          | MemGPT / Agent                  |
|-----------------------------|---------------------------------|
| RAM                         | Main Context                    |
| Disk                        | External Context                |
| Page In / Out               | memory function call 读写 memory  |
| Eviction / Page Replacement | queue eviction / recall storage |

MemGPT 架构：

| 部分                  | 作用         |
|---------------------|------------|
| System Instructions | 定义行为       |
| Working Context     | 高价值信息      |
| FIFO Queue          | 滚动消息历史     |
| Recall Storage      | 过去对话历史     |
| Archival Storage    | 长文档、大规模资料库 |

| 部分                | 作用                          |
|-------------------|-----------------------------|
| Queue Manager     | 管理 FIFO / recall / overflow |
| Function Executor | 解析模型输出，执行 memory 读写或检索      |

MemGPT 总结：

- 让 LLM 像拥有更大上下文。
- main context / external context 多级管理。
- LLM 通过 function calls 主动管理 memory。

不足：

- FIFO 和 recursive summary 简单但不最优。
- 早期消息不一定不重要。
- summary 可能丢细节。
- topic mixing 会影响检索。
- 过度依赖 LLM 自己决定 search/update，弱模型不稳定。

## 14. MemoryOS

MemoryOS 目标：

建立统一、系统化的 memory management framework，解决长期对话中早期信息丢失、跨会话事实不一致、个性化不足等问题。

核心：STM / MTM / LPM 三层 memory hierarchy。

| 层   | 全称                       | 保存什么                               |
|-----|--------------------------|------------------------------------|
| STM | Short-Term Memory        | 实时对话数据，基本单位是 dialogue page         |
| MTM | Mid-Term Memory          | 按 topic segment 组织的 dialogue pages |
| LPM | Long-Term Persona Memory | 用户画像、知识库、traits、agent persona      |

MemoryOS 关键机制：

| 机制             | 含义        |
|----------------|-----------|
| dialogue page  | 对话数据基本单位  |
| dialogue chain | 保持局部上下文连续 |

| 机制              | 含义                     |
|-----------------|------------------------|
| segment         | 相同 topic 的 pages       |
| segment summary | 对一组 pages 的压缩          |
| heat            | 由访问次数、交互活跃度、recency 计算 |

数据迁移：

STM --FIFO--> MTM --heat promotion--> LPM

Retrieval：

- STM：最近上下文，全量取回。
- MTM：两阶段检索：segment-level top-m，再 page-level top-k。
- LPM：检索 persona 信息，用户 profile/traits 等长期信息参与回答。

## 15. Mem0

Mem0 定位：

面向生产的长期 memory service，更关注 accuracy、latency、token cost、scalability、deployability。

核心做法：

| 概念               | 含义                             |
|------------------|--------------------------------|
| Message Pair     | 用户输入 + 模型输出作为 interaction unit |
| candidate fact   | 从新交互中抽取可能值得记忆的事实               |
| similar memories | 在 memory database 中检索相似旧记忆     |
| memory operation | 让 LLM 决定如何更新 memory            |

Memory operation：

ADD / UPDATE / DELETE / NOOP

一句话：

Mem0 把 memory update 看成数据库一致性维护问题：新事实来了以后，要判断是新增、更新、删除还是不操作。

MemGPT / MemoryOS / Mem0 对比：

| 维度          | MemGPT                   | MemoryOS              | Mem0                           |
|-------------|--------------------------|-----------------------|--------------------------------|
| 核心          | virtual context          | memory OS             | production memory service      |
| Swap        | main/external context 搬运 | STM->MTM->LPM         | conversation -> facts          |
| Compression | recursive summary        | topic segment summary | salient facts                  |
| Retrieve    | LLM 主动 search            | STM/MTM/LPM 分层检索      | dense vector / graph retrieval |
| Evict       | queue eviction           | heat 淘汰/晋升            | ADD/UPDATE/DELETE/NOOI         |

16. 自我进化：Learning Loop

Long-run Agent 不能只是记忆，还要自我改进。

Hermes Learning Loop：

| 步骤       | 含义                 |
|----------|--------------------|
| 整理记忆     | 从对话中提取值得记住的事实      |
| 创建 skill | 重复模式自动提取为 skill 文件 |
| 优化 skill | 发现现有 skill 错了就修正   |
| FTS5 召回  | 下次遇到类似话题，检索相关历史    |

课件问题：

- 反复犯的错、重复的低效行为？
- 主动拓展自己的 tool list？
- 压缩上下文，建立高效索引知识库？
- 定期反思和自我总结？
- 知识的迁移、泛化性？

考场句：

Self-improvement 让 Agent 把一次性经验沉淀为 memory 或 skill，下次遇到相似任务时能检索、复用并修正旧流程。

# 17. SubAgent / MultiAgent

课件位置：Long-run Agent 的 Harness Engineering。

| 概念         | 含义                         |
|------------|----------------------------|
| SubAgent   | 根据任务计划，把子任务下发给子 agent 并行完成 |
| MultiAgent | 多个 agent 团队协作，从工作看板领取任务    |
| Tool 治理    | 不是所有工具都给 agent 用，需要黑/白名单   |
| Context 治理 | 持久稳定运行的基石                  |
| 硬性约束       | 不能只靠 prompt 软约束，要靠系统约束     |

SubAgent 适用：

- 子任务边界清晰。
- 输入输出明确。
- 需要并行或复核。
- 主 agent 不想塞太多局部细节。

风险：

- token 成本上升。
- 角色不清会互相推诿。
- 子 agent 错误会污染主流程。
- 需要明确停止条件和输出格式。

# 18. 简答模板

## 18.1 Agent 是什么

Agent = LLM + Harness。LLM 负责理解、推理和生成行动意图；harness 负责 loop、tools、memory、skill、权限、执行和反馈。Agent 围绕 goal 多轮行动，而不是只生成一次回答。

## 18.2 ChatBot / Copilot / Agent 的区别

ChatBot 输入一句问题、输出一段回答；Copilot 根据局部上下文补全代码；Agent 输入一个任务，利用完整项目上下文、tool 和 loop 交付执行过程与结果。

## 18.3 Tab / IDE / CLI Agent 的区别

Tab Complete 适合局部、低风险补全；IDE Agent 适合仓库内小功能、重构和 bug 修复；CLI Agent 权限更大，能读写文件、跑命令并完成仓库级任务。

## 18.4 Tool / Provider / Model 怎么区分

Tool 是 Agent 产品或运行环境，Provider 是 API/计费/路由平台，Model 是实际推理生成的模型。三者分别对应交互工具、服务提供方和智能核心。

## 18.5 为什么需要 Tool

LLM 只能产生行动意图，工具让 Agent 访问文件、运行命令、调用外部服务并改变环境。工具结果作为 observation 回填，帮助下一轮推理并减少幻觉。

## 18.6 Skill 是什么

Skill 是把某个领域的流程、知识和 checklist 封装成 `skill.md`，在需要时按需加载。它避免把所有知识塞进 prompt，并让 Agent 对重复任务有稳定流程。

## 18.7 Long-run Agent 是什么

Long-run Agent 从 reactive 变成 proactive，不需要人一直坐在旁边。它通过 cron/heartbeat 定期醒来，通过 gateway 接收外部消息，通过 persistent memory 保存长期状态，并通过 learning loop 自我改进。

## 18.8 Memory 和 Context 的区别

Context 是本轮推理可见的信息，容量有限；Memory 是跨会话保存的持久信息，容量更大，但需要检索后放回 context。没有 memory，Long-run Agent 会反复忘记用户偏好、任务状态和历史经验。

## 18.9 Memory 管理五问

持久化 memory 要解决 Storage、Write、Retrieval、Update、Evict：写到哪里，什么时候写，什么时候读，如何合并纠错，以及空间或 context 满了怎么处理。

## 18.10 MemGPT / MemoryOS / Mem0 如何理解

MemGPT 用 virtual context management 让 LLM 像拥有更大上下文；MemoryOS 用 STM/MTM/LPM 建立分层 memory system；Mem0 面向生产，把 memory update 看成 ADD/UPDATE/DELETE/NOOP 的一致性维护。

## 18.11 Self-improvement 有什么用

Self-improvement 把对话中的重要事实写入 memory，把重复模式沉淀成 skill，并在发现旧 skill 错误时修正，使 Agent 下次遇到类似任务能复用经验、减少重复犯错。

## 18.12 SubAgent / MultiAgent 有什么用

SubAgent 把边界清晰的子任务下发出去并行完成，主 agent 汇总结果；MultiAgent 则让多个 agent 团队协作。关键是角色清楚、输入输出明确、能通信和校验。

# 19. 高频词

- Agent = LLM + Harness
- Loop / Tool / Skill
- ChatBot Answer -> Copilot Suggest -> Agent Act
- Tab Complete -> IDE Agent -> CLI Agent
- Tool / Provider / Model
- Goal / Context / Constraints / Output / Verification
- reactive -> proactive
- Cron / Heartbeat / Gateway
- Persistent Memory
- Storage / Write / Retrieval / Update / Evict
- Always inject / Keyword / Vector / Graph / Hybrid
- Sliding Window / Recency / Frequency / Relevance / Heat
- MemGPT: virtual context management
- MemoryOS: STM / MTM / LPM
- Mem0: ADD / UPDATE / DELETE / NOOP
- Learning Loop / Self Improvement
- SubAgent / MultiAgent

## 20. 考场最后 20 秒

1. 总纲: Agent = LLM + Harness
2. Code Agent: ChatBot -> Copilot -> Agent Act
3. 形态: Tab -> IDE -> CLI
4. 三层: Tool / Provider / Model
5. Prompt: Goal / Context / Constraints / Output / Verification
6. Long-run: reactive -> proactive
7. 时序: Cron / Heartbeat / Gateway
8. Memory: Context 有限, Memory 持久
9. 五问: Storage / Write / Retrieval / Update / Evict
10. 三系统: MemGPT / MemoryOS / Mem0